数据结构 · 第 1 章绪论 · 康奈尔听课笔记

1.2 课后习题：算法和算法评价

王道书 §1.2.3 单项选择题 01–18 · §1.2.4 答案与解析 · 习题讲解字幕 P2–P5

目录 · 点击标题跳转

这组题怎么用 . . . 1
一、核心方法：表格法数执行次数 . . . 2
二、概念题 . . . 2
三、单层循环 . . . 4
四、多层循环 . . . 8
五、递归 . . . 12
六、知识回顾：看代码秒判复杂度 . . . 14

这组题怎么用

↑回到目录

Q 18 题分几类？哪类最重要？

18 道选择题分三类：前 5 道是概念题，对应正课 1.2.1 和 1.2.3；后 13 道全是**求时间复杂度**，其中 7 道是统考真题。老师说“时间复杂度这类题是本章最重要的习题”，而且真题里基本每两年考一次。所以下面按题型分组，不按题号顺序。

类型	题号	答案
概念：特性、效率、复杂度含义	01 02 03 04 05	B C C B D
单层循环	06 07 12[2011] 15[2017] 16[2019]	D C A B B
多层循环	08 09 10 14[2014] 17[2022] 18[2025]	D A A C B B
递归	11 13[2012]	C B

勘误 / 说明

上标是统考真题年份，已逐一对照你放进来的 2011–2025 真题和解析核对过。

名词统一：题目里的“频度”（08 题）就是正课说的**执行次数**；老师说的“基本语句”“关键指令”和正课的**基本操作**是一回事。下文统一用“执行次数”和“基本操作”。

勘误 / 说明

这几段习题讲解视频都不用补看。老师讲题全靠“画表格”，这个方法和每道题的表格下面都整理出来了。习题讲解视频里唯一超出书的内容是 11 题的一个改编题（两次递归调用），也已收进笔记。老师在 18 题结尾推荐了一个“时间复杂度专题习题讲解视频”，它不在你的习题讲解文件夹里，看不看都行，方法和这里一致。

一、核心方法：表格法数执行次数

↑回到目录

Q 老师的“表格法”有哪几列？最后一步做什么？

老师在 06 题专门讲了一套“笨但不会错”的方法，后面所有循环题、递归题都用它。简单题可以一眼看出，但 2022、2025 这类题靠一眼容易错。

定义

表格法（单层循环 / 递归画两列，双层循环画三列）：

1. **第一列写循环变量（或递归参数）的取值**，从初值开始，按代码的变化规律写，最后一行设为未知量（比如 2^c），表示最后一次进入循环的那个值。
2. **（双层循环）第二列写内层变量的取值范围。**
3. **最后一列写这一行的执行次数**，全部加起来就是总执行次数 T。
4. **用“最后一次能进、下一次进不去”列不等式**，求出 c 和 n 的关系，代回 T，只保留最高阶、去系数。

回顾正课

这就是正课 1.2.2 “知识回顾”里三步法的展开版：①找最深层循环的一个基本操作；②求它的执行次数 $x = f(n)$；③取 $O(x)$。表格法只是把第②步写得一步不漏。

我的补充

最后一步不用真解不等式。老师在 15 题特别提醒：时间复杂度只关心数量级，所以列出 $c^2 \approx n$ 这种关系后，直接去掉系数和低阶项读出 $c \approx \sqrt{n}$ 就行。解方程配方虽然严谨，但“不像高手”，考场上浪费时间。

二、概念题

↑回到目录

Q 算法的五个特性 vs 好算法的标准?

01

一个算法应该具有（　）等重要特性。

A. 可维护性、可读性和可行性　　B. 可行性、确定性和有穷性

C. 确定性、有穷性和可靠性　　D. 可读性、正确性和可行性

答案：B

视频讲解

要区分两个概念。**特性**是“是不是算法”的门槛，五个缺一不可，就像判断是不是人要看有没有眼睛鼻子；**评价标准**是“好不好”的角度，比如正确性、可读性、健壮性、高效率与低存储量需求。A、C、D 里混进了可维护性、可读性、可靠性、正确性，都是评价标准。

回顾正课

正课 1.2.1 的知识回顾：**必须有**（五个特性）是有穷性、确定性、可行性、输入、输出；**最好有**（四个目标）是正确性、可读性、健壮性、高效率与低存储量需求。选项里出现“最好有”那一组的词，就不是特性。

Q 衡量时间效率看 CPU 时间吗？能不能拿空间换时间?

02

下列关于算法的说法中，正确的是（　）。

A. 算法的时间效率取决于算法执行所花的 CPU 时间

B. 在算法设计中不允许用牺牲空间效率的方式来换取好的时间效率

C. 算法必须具备有穷性、确定性等五个特性

D. 通常用时间效率和空间效率来衡量算法的优劣

答案：C

视频讲解

A 错：CPU 时间还取决于机器性能，用时间复杂度才能把机器的影响剥离掉，只看算法本身。B 错：时间要求高、空间要求低的场景，就该用空间换时间。D 错：时空效率只是“高效率”这一条，优劣还要看正确性、可读性、健壮性，说法太片面。

回顾正课

A 对应正课 1.2.2 开头“事后统计”的缺点：同一个算法换台机器、换种语言、换个编译器，跑出来的时间都不一样。

Q $O(n^2)$ 到底说的是什么和 n^2 的关系?

03

某算法的时间复杂度为 $O(n^2)$，则表示该算法的（　）。

A. 问题规模是 n^2　　B. 执行时间等于 n^2

C. 执行时间与 n^2 成正比　　D. 问题规模与 n^2 成正比

答案：C

视频讲解

最准确的说法是基本操作的执行次数与n^2同数量级，四个选项里没有，C 最接近。但 C 说“执行时间”也不太妥，因为时间复杂度不考虑具体机器。问题规模就是输入的

n，不是 n^2，所以 A、D 错。

书上解析

时间复杂度为 $O(n^2)$，说明 $T(n) \le cn^2$（c 为比例常数）。问题规模仍然是 n 而不是 n^2。

我的补充

C 还有第二处不严谨，老师没提：大 O 是**上界**，$T(n) \le cn^2$ 只说明“不超过 n^2 的常数倍”，并不保证“成正比”。正课 1.2.2 讲过同阶和上界的区别，考研里按“同阶”理解即可，遇到这种题选最接近的。

Q $O(1)$ 空间 = 不用空间？辅助空间和总空间的区别？

04

若某算法的空间复杂度为 $O(1)$，则表示该算法（　）。

A. 不需要任何辅助空间　　B. 所需辅助空间大小与问题规模 n 无关

C. 不需要任何空间　　D. 所需空间大小与问题规模 n 无关

答案：B

视频讲解

空间复杂度只看**额外开辟的辅助空间**。定义一个 `int i`，或者 100 个、1000 个变量，只要和 n 无关，都是 $O(1)$；开 `a[n]` 就是 $O(n)$，开 `a[n][n]` 就是 $O(n^2)$。A、C 错，因为循环变量本身就是辅助空间，程序指令也要占空间。D 错在“空间”和“辅助空间”一字之差：如果数组 `a[n]` 是作为参数传进来的，总空间和 n 有关，但辅助空间仍可能是 $O(1)$。

回顾正课

正课 1.2.3 的王道书表格：指令和常数不计、**输入数据不计**、只计辅助空间。D 就是故意把“输入数据”混进来。

Q 比较复杂度时，系数和低阶项怎么处理？

05

下列关于时间复杂度的函数中，时间复杂度最小的是（　）。

A. $T_1(n) = n\log_2 n + 5000n$　　B. $T_2(n) = n^2 - 8000n$

C. $T_3(n) = n\log_2 n - 6000n$　　D. $T_4(n) = 20000\log_2 n$

答案：D

视频讲解

抓大放小：只看最高阶，系数统统扔掉。A、C 是 $O(n\log_2 n)$，因为 $n \to \infty$ 时 $\log_2 n$ 远大于 5000；B 是 $O(n^2)$；D 的系数 20000 再大也只是常数，是 $O(\log_2 n)$，最小。

我的补充

陷阱就是 D 前面的 20000：小 n 时它确实比 A 大，但复杂度只看 $n \to \infty$。正课 1.2.2 第七节“小故事”讲的就是这一点。

三、单层循环

↑回到目录

Q 变量每次乘 2，循环几次?

06

下列算法的时间复杂度为（　）。

```
void fun(int n){
    int i=1;
    while(i<=n)
        i=i*2;
}
```

A. $O(n)$　　B. $O(n^2)$　　C. $O(n\log_2 n)$　　D. $O(\log_2 n)$

答案：D

循环变量 i	$1=2^0$	2^1	2^2	$\cdots$	2^c（最后一次进入）
执行次数	1	1	1	$\cdots$	1

共 $c+1$ 个 1。最后一次能进：$2^c \le n$；下一次进不去：$2^{c+1} > n$。所以 $c \le \log_2 n < c+1$，即 $c = \lfloor \log_2 n \rfloor$，$T = \lfloor \log_2 n \rfloor + 1 = O(\log_2 n)$。

视频讲解

这题一眼就能看出“每次乘 2 就是对数”，老师故意用表格法慢慢做，是为了练方法：简单题觉得绕很正常，2022、2025 那种题才用得上。

回顾正课

就是正课 1.2.2 的“小练习 1：指数递增型爱你”，当时算出精确值也是 $\lfloor \log_2 n \rfloor + 1$。

Q 条件是 $i^3 \le n$，循环几次?

07

下列算法的时间复杂度为（　）。

```
void fun(int n){
    int i=0;
    while(i*i*i<=n)
        i++;
}
```

A. $O(n)$　　B. $O(n\log_2 n)$　　C. $O(\sqrt[3]{n})$　　D. $O(\sqrt{n})$

答案：C

循环变量 i	0	1	2	$\cdots$	c（最后一次进入）
执行次数	1	1	1	$\cdots$	1

共 $c+1$ 次。$c^3 \le n < (c+1)^3$，开三次方得 $c \le \sqrt[3]{n} < c+1$，所以 $T = \lfloor \sqrt[3]{n} \rfloor + 1 = O(\sqrt[3]{n})$。

Q x 从 2 开始、条件是 x<n/2，会改变数量级吗?

12 · 2011 统考真题

设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是（　）。

```
x=2;
while(x<n/2)
    x=2*x;
```

A. $O(\log_2 n)$　　B. $O(n)$　　C. $O(n \log_2 n)$　　D. $O(n^2)$

答案：A

循环变量 x	2^1	2^2	$\cdots$	2^c（最后一次进入）
执行次数	1	1	$\cdots$	1

这次从 2^1 开始，共 c 个 1。$2^c < n/2 \le 2^{c+1}$，取对数得 $c < \log_2 n - 1 \le c+1$，即 $\log_2 n - 2 \le c < \log_2 n - 1$，$T = O(\log_2 n)$。

我的补充

和 06 题比只改了两处：初值从 1 变 2，上界从 n 变 $n/2$。这两处都只让次数差一两次，不改变数量级。真题常用这种“换个初值、换个边界”的方式出题，看清“每次乘 2”就够了。

Q sum += ++i 的 sum 怎么增长?

15 · 2017 统考真题

下列函数的时间复杂度是（　）。

```
int func(int n){
    int i=0, sum=0;
    while(sum<n) sum += ++i;
    return i;
}
```

A. $O(\log_2 n)$　　B. $O(n^{1/2})$　　C. $O(n)$　　D. $O(n \log_2 n)$

答案：B

`sum += ++i` 拆成两句：先 `i=i+1`，再 `sum=sum+i`。循环变量是 sum，逐步追踪（以 $n=10$ 为例）：

第几次进入	进入前 sum	判断 sum<10	执行后 i	执行后 sum
1	0	进	1	1
2	1	进	2	3
3	3	进	3	6
4	6	进	4	10
—	10	不进，结束		

执行 k 次后 sum $=1+2+\cdots+k=\frac{k(k+1)}{2}$。循环在 sum 第一次 $\geq n$ 时停，所以 $\frac{k(k+1)}{2}\approx n$，即 $k^2\approx 2n$，$k\approx\sqrt{2n}$，$T=O(n^{1/2})$。

视频讲解

可以老老实实配方解不等式 $\frac{k(k+1)}{2}<n\leq\frac{(k+1)(k+2)}{2}$，但高手做法是：两边都是 k^2 量级，直接看成 $n\approx k^2$，马上得 $k\approx\sqrt{n}$。

Q 条件 $(x+1)^2\leq n$，循环几次?

16 · 2019 统考真题

设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是（　）。

```
x=0;
while(n>=(x+1)*(x+1))
    x=x+1;
```

A. $O(\log_2 n)$　　B. $O(n^{1/2})$　　C. $O(n)$　　D. $O(n^2)$

答案：B

循环变量 x	0	1	2	$\cdots$	c（最后一次进入）
执行次数	1	1	1	$\cdots$	1

共 $c+1$ 次。$(c+1)^2\leq n<(c+2)^2$，开方得 $c+1\leq\sqrt{n}<c+2$，所以 $T=c+1=\lfloor\sqrt{n}\rfloor=O(\sqrt{n})$。

我的补充

06、07、15、16 四题其实是一个模板：**循环变量线性 +1，条件里是变量的 k 次方 $\leq n$，次数就是 $\sqrt[k]{n}$**；变量每次乘 2，次数就是 $\log_2 n$。

四、多层循环

↑回到目录

Q 内层次数依赖外层时，表格要多一列什么？

08

某个程序段如下：

```
for(i=n-1;i>1;i--)
    for(j=1;j<i;j++)
        if(A[j]>A[j+1])
            A[j]与 A[j+1]对换;
```

其中 n 为正整数，则最后一行语句的频度在最坏情况下是（　）。

A. $O(n)$　　B. $O(n\log_2 n)$　　C. $O(n^3)$　　D. $O(n^2)$

答案：D

“频度”就是执行次数；“最坏情况”就是每次 `if` 都成立，交换语句每轮都执行。于是问题变成：双层循环一共转多少次。

外层 i	内层 j 的取值	执行次数
$n-1$	$1\sim n-2$	$n-2$
$n-2$	$1\sim n-3$	$n-3$
$\cdots$	$\cdots$	$\cdots$
2	1	1

$$T=1+2+\cdots+(n-2)=\frac{(n-1)(n-2)}{2}=O(n^2)。$$

勘误 / 说明

习题讲解视频里老师把第一行的次数写成了 $n-1$，总和算成 $\frac{n(n-1)}{2}$，**这里算错了**：$i=n-1$ 时，j 从 1 到 $n-2$，只有 $n-2$ 次。正确结果是书上的 $\frac{(n-1)(n-2)}{2}$。数量级都是 $O(n^2)$，选择题不受影响，但如果题目问精确次数（像 10 题那样），就会选错。

我的补充

这段代码是**冒泡排序**，第 8 章会学。到时候记住结论：冒泡排序最坏情况下比较和交换都是 $O(n^2)$。

Q 有 if-else 两个分支时，复杂度取哪个?

09

下列程序段的时间复杂度为（　）。

```
if(n>=0){
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            printf(" 输入数据大于或等于零\n");
}
else{
    for(int j=0;j<n;j++)
        printf(" 输入数据小于零\n");
}
```

A. $O(n^2)$　　B. $O(n)$　　C. $O(1)$　　D. $O(n\log_2 n)$

答案：A

if 分支是 n 行、每行 n 次，共 n^2 次；else 分支是 n 次。**分支语句取复杂度最高的那个分支**，所以是 $O(n^2)$。

视频讲解

有同学会说“$n<0$ 时 else 里的循环根本不执行，这题有问题”。这是误解：分析复杂度时只看代码结构，不代入 n 的具体值。每个分支分别算，取最大。

Q 题目问“执行次数”而不是“复杂度”时，要算到多精确?

10

下列算法中加下划线的语句（m++）的执行次数为（　）。

```
int m=0,i,j;
for(i=1;i<=n;i++)
    for(j=1;j<=2*i;j++)
        m++;
```

A. $n(n+1)$　　B. n　　C. $n+1$　　D. n^2

答案：A

外层 i	内层 j 的取值	执行次数
1	$1\sim 2$	2
2	$1\sim 4$	4
$\cdots$	$\cdots$	$\cdots$
n	$1\sim 2n$	$2n$

$T=2+4+\cdots+2n=\dfrac{(2+2n)\cdot n}{2}=n(n+1)$。

易错提醒

问的是**精确次数**，不能只写数量级。D 的 n^2 和 A 同阶，就是专门给只算数量级的人准备的。

Q 外层乘 2、内层固定 n 次，总次数怎么算?

14 ・2014 统考真题

下列程序段的时间复杂度是（　）。

```
count=0;
for(k=1;k<=n;k*=2)
    for(j=1;j<=n;j++)
        count++;
```

A. $O(\log_2 n)$　　B. $O(n)$　　C. $O(n\log_2 n)$　　D. $O(n^2)$

答案：C

外层 k	内层 j 的取值	执行次数
$1=2^0$	$1\sim n$	n
2^1	$1\sim n$	n
$\cdots$	$\cdots$	$\cdots$
2^c（最后一次进入）	$1\sim n$	n

内层和 k 无关，每行都是 n。外层行数同 06 题：$c=\lfloor\log_2 n\rfloor$，共 $c+1$ 行。$T=n(\lfloor\log_2 n\rfloor+1)=O(n\log_2 n)$。

我的补充

内外层**互不依赖**时直接用乘法规则：外层 $O(\log_2 n)$ 乘内层 $O(n)$。17、18 题内层依赖外层，就不能直接乘了。

Q 外层乘 2、内层跑 i 次，为什么不是 $n\log n$?

17 ・2022 统考真题

下列程序段的时间复杂度是（　）。

```
int sum=0;
for(int i=1;i<n;i*=2)
    for(int j=0;j<i;j++)
        sum++;
```

A. $O(\log_2 n)$　　B. $O(n)$　　C. $O(n\log_2 n)$　　D. $O(n^2)$

答案：B

外层 i	内层 j 的取值	执行次数
$1=2^0$	0	1
2^1	$0\sim1$	2
2^2	$0\sim3$	4
$\cdots$	$\cdots$	$\cdots$
2^t（最后一次进入）	$0\sim2^t-1$	2^t

$T=1+2+4+\cdots+2^t=2^{t+1}-1$（等比数列）。由 $2^t<n\le2^{t+1}$ 得 $n-1\le T<2n-1$，所以 $T=O(n)$。以 $n=10$ 验证：i 取 1、2、4、8，$T=1+2+4+8=15$，确实在 $9\sim19$ 之间。

易错提醒

最容易错选 C：看到外层 $\log_2 n$ 次、内层“最多 n 次”就直接相乘。乘法规则只给出上界 $O(n\log_2 n)$，但实际上内层次数是 1、2、4…，前面各行加起来还不到最后一行，等比数列的和和最后一项同阶，所以是 $O(n)$。

勘误 / 说明

书上解析写“对于单层循环如 `for(j=0;j<=i;j++) sum++;`，可以直接数出执行次数为 i”，这里 `j<=i` 是笔误，应为 `j<i`（题目代码就是 `j<i`），`j<=i` 会执行 $i+1$ 次。

Q 外层到 $\sqrt{n}$、内层跑 i 次，总次数?

18 · 2025 统考真题

下列程序段的时间复杂度是（　）。

```
int count=0,i,j;
for(i=1;i*i<=n;i++)
    for(j=1;j<=i;j++)
        count++;
```

A. $O(\log_2 n)$　　B. $O(n)$　　C. $O(n\log_2 n)$　　D. $O(n^2)$

答案：B

外层条件 $i^2\le n$，所以 i 从 1 到 $m=\lfloor\sqrt{n}\rfloor$。

外层 i	内层 j 的取值	执行次数
1	1	1
2	$1\sim2$	2
$\cdots$	$\cdots$	$\cdots$
$m=\lfloor\sqrt{n}\rfloor$	$1\sim m$	m

$T=1+2+\cdots+m=\dfrac{m(m+1)}{2}\approx\dfrac{n}{2}+\dfrac{\sqrt{n}}{2}$，最高阶是 $\frac{n}{2}$，去掉系数得 $O(n)$。

勘误 / 说明

书上和习题讲解视频都写成 $T=\frac{n}{2}+\frac{\lfloor\sqrt{n}\rfloor}{2}$ 并用了等号，这只在 n 是完全平方数时成立。精确写法是 $\frac{\lfloor\sqrt{n}\rfloor(\lfloor\sqrt{n}\rfloor+1)}{2}$，你那份 2025 真题官方解析写的就是 $\frac{\sqrt{n}(\sqrt{n}+1)}{2}$。不影响 $O(n)$。

视频讲解

18 题的讲解换了一位老师，最后总结了**两类循环题**（习题讲解视频叫类型一、类型二，下文第六节改叫 +1 型、非 +1 型）：

- **+1 型（习题讲解视频的类型一）：每层都是 i++、j++ 这种 +1 的。**求出每层的最大取值范围，直接相乘。内层依赖外层（如 `j<i`）时，把 i 放大成 n 再乘，结果的阶不变。
- **非 +1 型（习题讲解视频的类型二）：有 i*=2 这类非 +1 变化的。**不能套公式，要从外到内一层层拆，列出外层的取值，再算每个取值下内层执行几次，最后求和。

我的补充

+1 型的“放大成 n”给出的是上界，之所以阶不变，是因为 $j<i$ 这种依赖是线性的，$1+2+\cdots+n$ 和 $n\cdot n$ 同为 n^2 阶。17 题就是反例：外层乘 2 属于非 +1 型，直接放大会得到错误的 $n\log_2 n$。

五、递归

↑回到目录

Q 递归的时间复杂度看什么？2*Func(n/2)+n 里的 2 和 +n 要不要管？

11

下列函数代码的时间复杂度是（　）。

```
int Func(int n){
    if(n==1) return 1;
    else return 2*Func(n/2)+n;
}
```

A. $O(n)$　　B. $O(n\log_2 n)$　　C. $O(\log_2 n)$　　D. $O(n^2)$

答案：C

递归的时间复杂度取决于**一共调用了多少次递归函数**，每次调用里只做常数次操作。表格第一列改成递归参数：

递归参数 i	n	$n/2$	$n/4$	$\cdots$	1
调用次数	1	1	1	$\cdots$	1

设 $n=2^c$，参数从 2^c 一路减半到 2^0，共 $c+1=\log_2 n+1$ 次调用，$T=O(\log_2 n)$。以

$n=8$ 为例：调用 Func(8)、Func(4)、Func(2)、Func(1)，共 4 次。

视频讲解

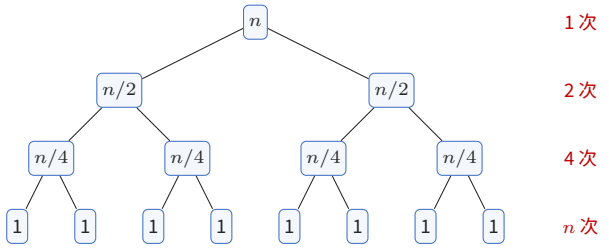
式子里的系数 2 和“$+n$”都不影响调用次数：参数传进来后 n 就是个常数，改成 $+1$、$3\times$ 都一样，所以 2* 和 +n 都可以忽略。

易错提醒

“$+n$”是一次**加法**，不是一个循环，不会让每层做 n 次操作。别看到 n 就以为是 $O(n\log_2 n)$。

习题讲解视频里的改编题：把返回语句改成 `return Func(n/2)*Func(n/2);`（不做编译优化），复杂度变成多少？

关键区别是：每层要调用**两次**下一层，调用次数按层翻倍，画出来是一棵满二叉树：



递归参数 i	n	$n/2$	$n/4$	$\cdots$	1
调用次数	1	2	4	$\cdots$	n

设 $n=2^c$，$T=1+2+4+\cdots+2^c=2^{c+1}-1=2n-1$，复杂度变成 $O(n)$。所以递归题先看**每层调用自己几次**：一次是一条链，两次是一棵树。

Q n*fact(n-1) 调用几次?

13 • 2012 统考真题

求整数 n（$n\geq 0$）的阶乘的算法如下，其时间复杂度是（　）。

```
int fact(int n){
    if(n<=1) return 1;
    return n*fact(n-1);
}
```

A. $O(\log_2 n)$　　B. $O(n)$　　C. $O(n\log_2 n)$　　D. $O(n^2)$

答案：B

递归参数 i	n	$n-1$	$n-2$	$\cdots$	1
调用次数	1	1	1	$\cdots$	1

参数每次减 1，从 n 到 1 共 n 次调用，$T=O(n)$。乘号前的 n 和 11 题的系数一样，不影响调用次数。

回顾正课

正课 1.2.3 的“递归程序”：这段代码的空间复杂度也是 $O(n)$，因为递归深度为 n，函数调用栈要压 n 层。时间看调用总次数，空间看调用深度，单链递归两者相等；11 题的改编题时间是 $O(n)$，但深度只有 $\log_2 n$，空间是 $O(\log_2 n)$。

勘误 / 说明

书上 11、13 题的解析写“每次递归执行的基本语句是 `if(n==1) return 1;`“，这个说法有点别扭。实际意思是：每次调用只做一次判断和常数次运算，所以总次数 = 调用次数 × 常数。按”数调用次数”理解即可。

六、知识回顾：看代码秒判复杂度

↑回到目录

Q 常见循环 / 递归写法各是什么复杂度?

做完 13 道题，把每道题的循环变量**从哪开始、每次怎么变、到哪停**都摊开放在一起，就能看出复杂度是怎么来的。先统一两个叫法：

勘误 / 说明

下面按习题讲解视频（18 题）的两类来分，改叫 +1 型、非 +1 型。

更正正课 1.2.2 第六节：那里的“类型 1 / 类型 2”是我起的名字，书上（第 11–12 页“归纳总结”）只编号 1、2，分法也和这里不同。

- **+1 型**：每层循环变量都是 `i++`、`i--` 这种每次变 1 的。做法：每层取最大范围，相乘；内层依赖外层（如 `j<i`）时把它放大到上界再乘，阶不变。
- **非 +1 型**：至少一层是 `i*=2` 这类，或者判断的变量每次加的量在变（如 15 题的 sum）。做法：不能直接乘，要列出外层每个取值，算出每轮内层次数再求和。

1. 单层循环（c 表示最后一次进入循环时的指数或变量值）

题	变量：初值，每次怎么变	循环条件	类型	列不等式 → 执行次数	复杂度
09 else	j 从 0，每次 +1	`j<n`	+1	$j=0\sim n-1$ → n 次	$O(n)$
07	i 从 0，每次 +1	`i*i*i<=n`	+1	$c^3\le n<(c+1)^3$ → $\lfloor\sqrt[3]{n}\rfloor+1$ 次	$O(\sqrt[3]{n})$
16	x 从 0，每次 +1	`n>=(x+1)*(x+1)`	+1	$(c+1)^2\le n<(c+2)^2$ → $\lfloor\sqrt{n}\rfloor$ 次	$O(\sqrt{n})$
06	i 从 1，每次 ×2	`i<=n`	非 +1	$2^c\le n<2^{c+1}$ → $\lfloor\log_2 n\rfloor+1$ 次	$O(\log_2 n)$
12	x 从 2，每次 ×2	`x<n/2`	非 +1	$2^c<\frac{n}{2}\le 2^{c+1}$ → 约 $\log_2 n-2$ 次	$O(\log_2 n)$

15	i 从 0 每次 +1；sum 从 0，每次加上新的 i	sum<n	非 +1	k 次后 sum $=\frac{k(k+1)}{2}$，到 $\geq n$ 停 → 约 $\sqrt{2n}$ 次	$O(\sqrt{n})$

规律：变量 +1、条件是变量的 k 次方 $\leq n$，次数就是 $\sqrt[k]{n}$；变量每次 ×2，次数就是 $\log_2 n$。初值和边界里的常数（如 12 题的 2 和 $n/2$）只差一两次，不改变阶。

2. 双层循环

题	外层：初值，变化，条件	内层：范围，是否依赖外层	类型	每轮次数 → 求和	复杂度
09 if	i 从 0，+1，i<n，共 n 轮	j 为 $0 \sim n-1$，不依赖	+1	每轮 n → $n \cdot n = n^2$	$O(n^2)$
10	i 从 1，+1，i<=n，共 n 轮	j 为 $1 \sim 2i$，依赖	+1	每轮 $2i$ → $2+4+\cdots+2n = n(n+1)$	$O(n^2)$
08	i 从 $n-1$，−1，i>1，共 $n-2$ 轮	j 为 $1 \sim i-1$，依赖	+1	每轮 $i-1$ → $1+2+\cdots+(n-2) = \frac{(n-1)(n-2)}{2}$	$O(n^2)$
18	i 从 1，+1，i*i<=n，共 $m = \lfloor\sqrt{n}\rfloor$ 轮	j 为 $1 \sim i$，依赖	+1	每轮 i → $1+2+\cdots+m = \frac{m(m+1)}{2} \approx \frac{n}{2}$	$O(n)$
14	k 从 1，×2，k<=n，共 $\lfloor\log_2 n\rfloor+1$ 轮	j 为 $1 \sim n$，不依赖	非 +1	每轮 n → $n(\lfloor\log_2 n\rfloor+1)$	$O(n\log_2 n)$
17	i 从 1，×2，i<n，取 $1,2,4,\cdots,2^t$	j 为 $0 \sim i-1$，依赖	非 +1	每轮 i → $1+2+\cdots+2^t = 2^{t+1}-1 < 2n$	$O(n)$

对照两种做法验算：

- **+1 型直接乘**：09 if、10、08 都是“外层约 n 轮 × 内层最多约 n 次” $= n^2$；18 是“外层 $\sqrt{n}$ 轮 × 内层最多 $\sqrt{n}$ 次” $= n$。和表里求和的结果同阶。
- **非 +1 型要看依赖**：14 内层不依赖外层，“$\log_2 n$ 轮 × 每轮 n”可以直接乘；17 内层依赖外层，直接乘得到的 $n\log_2 n$ 只是上界，**必须求和**，等比数列的和与最后一项同阶，结果是 $O(n)$。

3. 递归

题	参数每次怎么变	每次调用自己几次	每层调用次数 → 总调用次数	时间	空间（递归深度）
11	$n \to n/2$，到 1 停	1 次	每层 1 次，共 $\log_2 n + 1$ 层 → $\log_2 n + 1$	$O(\log_2 n)$	$O(\log_2 n)$
13	$n \to n-1$，到 1 停	1 次	每层 1 次，共 n 层 → n	$O(n)$	$O(n)$
11 改编	$n \to n/2$，到 1 停	2 次	每层 $1, 2, 4, \cdots, n$ 次 → $2n-1$	$O(n)$	$O(\log_2 n)$

调自己 1 次时，调用次数就等于递归深度，时间和空间同阶；调 2 次时，调用次数按层翻倍，时间变大，但深度不变，空间仍是层数。

考点

口诀（只给最容易混的三处，其余看表就能直接想到）：

1. **“乘二取对数，平方开根号”**：循环变量每次乘 2 → $\log_2 n$；条件里变量平方 $\le n$ → $\sqrt{n}$。
2. **“外层翻倍内跟随，等比求和是 n 阶”**：外层乘 2、内层跑 i 次（17 题），不是 $n \log n$。
3. **“递归数调用，一调一条链，二调一棵树”**：调自己 1 次看递归深度；调 2 次按层翻倍求和。

易错提醒

- 问“执行次数”要算精确值（10 题），问“复杂度”才只看阶。
- 分支语句取最大的分支，不代入 n 的具体值（09 题）。
- 内层依赖外层时不能无脑相乘，外层是乘 2 时尤其要列表求和（17 题）。

总结区・用自己的话复述这一节

这 18 题前 5 道考概念：五个特性是“必须有”，正确性、可读性等是“最好有”；时间效率看时间复杂度而不是 CPU 时间；$O(1)$ 空间指辅助空间与 n 无关，不是不用空间；比较复杂度只看最高阶、不看系数。

后 13 道都是求时间复杂度，统一用表格法：第一列写循环变量（或递归参数）的取值，最后一行设为未知量；最后一列写每行执行次数并求和；用“最后一次能进、下一次进不去”列不等式求出未知量和 n 的关系，去掉系数和低阶项。递归题数的是调用次数，先看每层调用自己几次。

自测：

1. `for(i=1;i<n;i*=3) for(j=0;j<i;j++) x++;` 的复杂度？
2. `i=1; while(i*i<n) i*=2;` 的复杂度？
3. 把 13 题改成 `return fact(n-1)+fact(n-1);`，时间复杂度和空间复杂度各是多少？